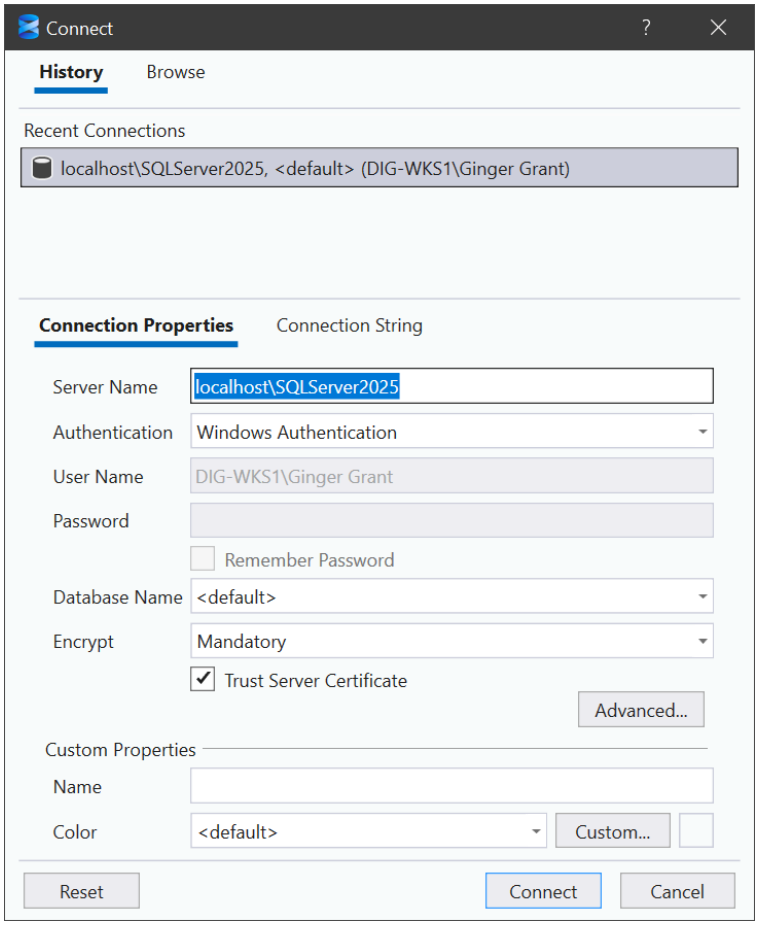
Exercise 1 – Introduction to SQL Server

1. Open up SQL Server Management Studio 22 and enter the name of the SQL Server 2025 you have installed on your desktop in the Server Name box. Set the authentication to Windows Authentication as shown here.



2. Select the database AdventureworksDW2020, which was included in the gitrepo so that you could load it to your local PC, then click on **New Query**
3. Validate that you are using SQL Server 2025 with this query

```
SELECT @@VERSION;
```

4. Check to see what version the database is

```
SELECT compatibility_level
FROM sys.databases
WHERE name = 'AdventureWorksDW2020'
```

5. If the column is not 170, and if you downloaded the version from github, it isn't, you will need to change the compatibility. Run the following command

```
ALTER DATABASE AdventureWorksDW2020
    SET COMPATIBILITY_LEVEL = 170;
```

Check the results again, and you will see a compatibility_level of 170.

```
SELECT compatibility_level
FROM sys.databases
WHERE name = 'AdventureWorksDW2020'
```

6. To ensure that the emails stored in the database are valid, run the the next query to validate the emails using REGEXP_LIKE a new T-SQL command

```
SELECT     CustomerKey,
     EmailAddress,
     CASE
         WHEN REGEXP_LIKE(EmailAddress, '^[A-Za-z0-9._%+\-
 ]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}$')
         THEN 'Valid'
         ELSE 'Invalid'
     END AS EmailValidation
 FROM dbo.DimCustomer
 WHERE EmailAddress IS NOT NULL
```

7. Review the results. To show only the invalid records, add this line to the end of the SQL statement

```
AND  NOT  REGEXP_LIKE(EmailAddress, '^[A-Za-z0-9._%+\-]+@[A-Za-z0-
9.\-]+\.[A-Za-z]{2,}$')
```

In the results, you will see the invalid email addresses.

8. Next let's examine the new JSON features in SQL Server 2025. First we are going to create a new table in the database using the new JSON data type.

```
CREATE TABLE dbo.CustomerProfiles (
    ProfileID INT PRIMARY KEY IDENTITY,
    CustomerKey INT NOT NULL,
    ProfileData JSON NOT NULL,  -- Native JSON type (NEW)
    CreatedDate DATETIME2 DEFAULT GETDATE()
);

GO
```

9. Next let's insert some data into the table

```
INSERT INTO dbo.CustomerProfiles (CustomerKey, ProfileData)
SELECT TOP (20)
    CustomerKey,
    JSON_OBJECT(
        'customerId' :  CustomerKey,
        'name'       :  CONCAT(FirstName, ' ', LastName),
        'email'      :  EmailAddress,
        'phone'      :  Phone
    ) AS ProfileData
FROM dbo.DimCustomer
WHERE FirstName IS NOT NULL;
```

10. Now let's query the JSON data

```
SELECT JSON_VALUE(ProfileData, '$.name')
FROM CustomerProfiles
WHERE JSON_VALUE(ProfileData, '$.email') LIKE '%adventure%';
```

End of Exercise

Exercise 2 – SSMS 22 and AI

Explore, Document, and Build with GitHub Copilot on SQL Server 2025

Explore the schema in Object Explorer

1. Expand **Databases** ▸ **AdventureWorksDW2020** ▸ **Tables**.
2. Note the twenty-five tables.These are separate from the greyed-out **System Databases** and system objects.
3. Peek at **Views** and **Programmability** ▸ **Stored Procedures** – you'll add one of each later in this lab.

Inspect metadata with catalog views

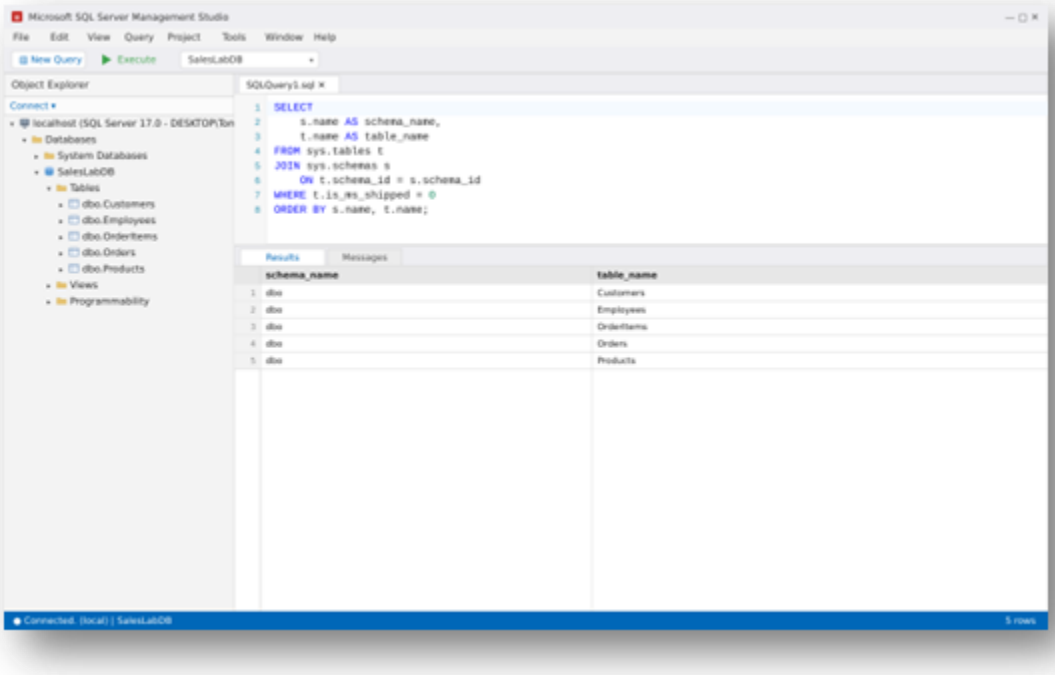
Clicking through Object Explorer is fine for a few tables, but catalog views scale and give you exactly the context Copilot needs. Click on **New Query** to create a query for each of the following queries.

4. List the user tables

```
SELECT
    s.name AS schema_name,
    t.name AS table_name
```

```
FROM sys.tables t
JOIN sys.schemas s
    ON t.schema_id = s.schema_id
WHERE t.is_ms_shipped = 0
ORDER BY s.name, t.name;
```

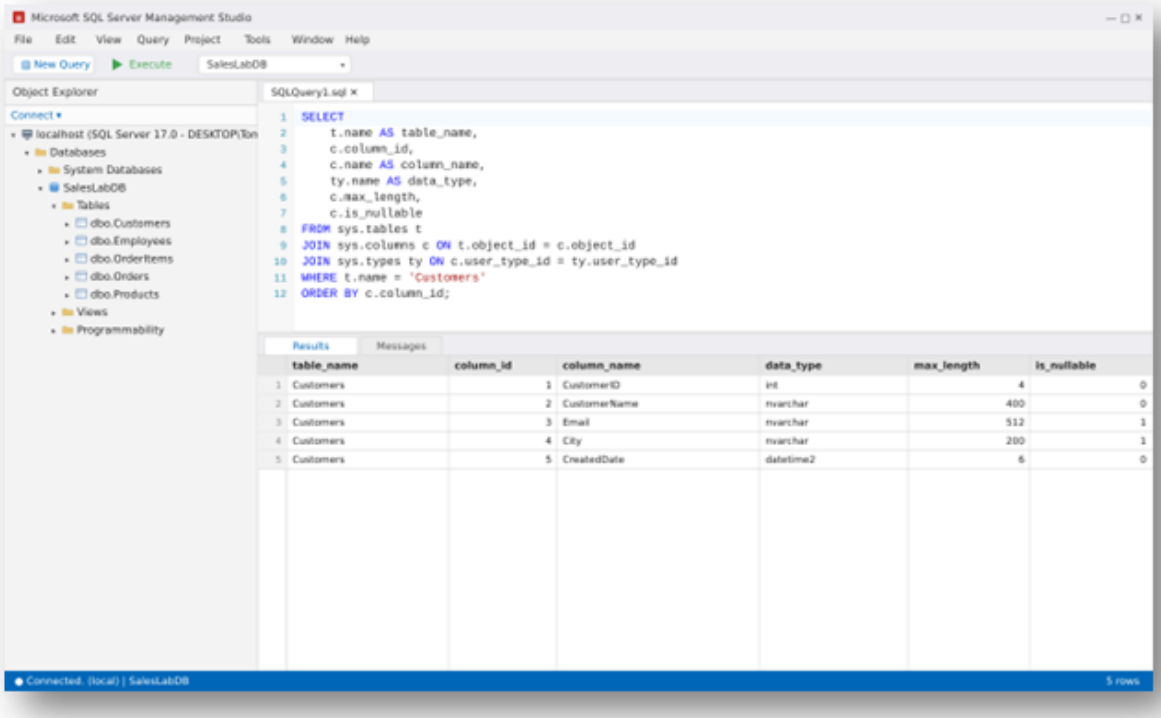
The results should look like this



5. Click on **New Query** to create a query to Inspect columns and data types by entering this query in the window

```
SELECT
    t.name  AS table_name,
    c.column_id,
    c.name  AS column_name,
    ty.name AS data_type,
    c.max_length,
    c.is_nullable
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
JOIN sys.types  ty ON c.user_type_id = ty.user_type_id
WHERE t.name = 'Customers'
ORDER BY c.column_id;
```

Here are the results you should expect



6. Find primary and foreign keys

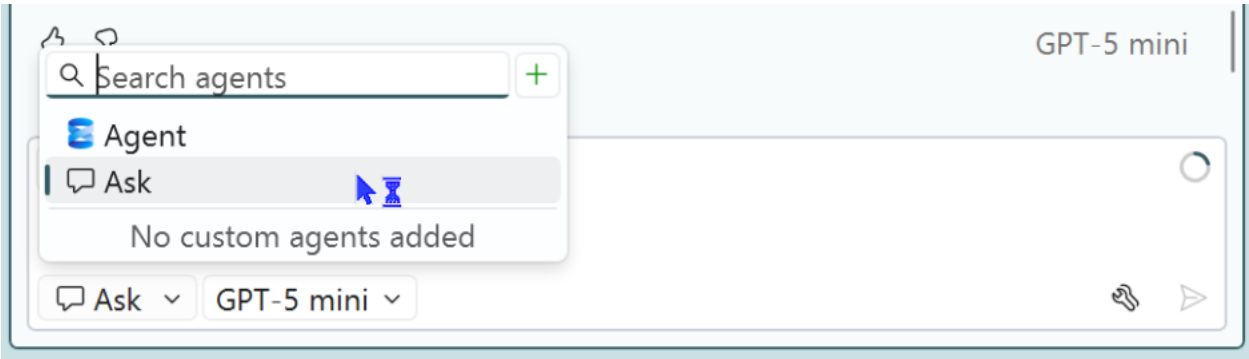
```
SELECT
    OBJECT_SCHEMA_NAME(parent_object_id) AS schema_name,
    OBJECT_NAME(parent_object_id)        AS table_name,
    name                                 AS constraint_name,
    type_desc
FROM sys.objects
WHERE type IN ('PK', 'F')  -- PRIMARY KEY and FOREIGN KEY
ORDER BY schema_name, table_name, type_desc;
```

7. Document your tables with GitHub Copilot (Ask mode)

SSMS 22 offers two Copilot modes. **Ask mode** answers questions and generates snippets – use it when you *need knowledge, are exploring, or want a quick answer*. **Agent mode** runs multi-step investigations and needs your approval for every query. Documentation is a perfect Ask-mode task.

Open the **GitHub Copilot** chat panel in SSMS 22

8. Set the mode to **Ask** by clicking on the icon in the bottom left side of the chat window as shown here



9. Open up a new query by clicking on the **New Query** button 3 times, once for each of the following queries and run each query

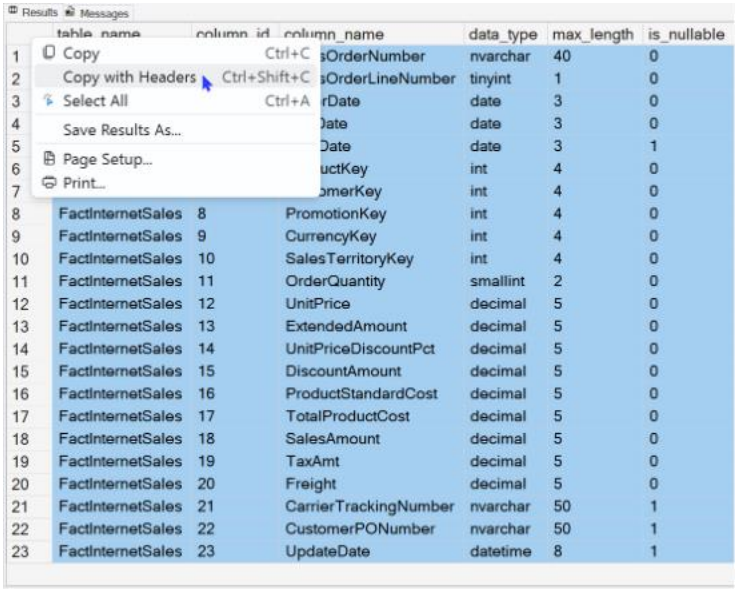
```
SELECT
    t.name AS table_name,
    c.column_id,
    c.name AS column_name,
    ty.name AS data_type,
    c.max_length,
    c.is_nullable
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id
WHERE t.name = 'FactInternetSales'
ORDER BY c.column_id;

SELECT
    t.name AS table_name,
    c.column_id,
    c.name AS column_name,
    ty.name AS data_type,
    c.max_length,
    c.is_nullable
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id
WHERE t.name = 'DimGeography'
ORDER BY c.column_id;

SELECT
    t.name AS table_name,
    c.column_id,
    c.name AS column_name,
    ty.name AS data_type,
    c.max_length,
```

```
    c.is_nullable
FROM sys.tables t
JOIN sys.columns c ON t.object_id = c.object_id
JOIN sys.types ty ON c.user_type_id = ty.user_type_id
WHERE t.name = 'DimProduct'
ORDER BY c.column_id;
```

10. Enter the following text into the GitHub Copliot chat window

```
Enter the following text into the GitHub Copliot chat window
Based on the following SQL Server schema metadata, write concise
documentation for each user table. For each table include:
- business purpose
- key columns and their meaning
- likely primary key and foreign key relationships
- developer usage notes
 Provide the results formatted as Markdown tables
```

11. Click on the corner of each of the results and select **Copy with Headers** as shown here



	table_name	column_id	column_name	data_type	max_length	is_nullable
1			sOrderNumber	nvarchar	40	0
2			sOrderLineNumber	tinyint	1	0
3			rDate	date	3	0
4			Date	date	3	0
5			Date	date	3	1
6			uctKey	int	4	0
7			omerKey	int	4	0
8	FactInternetSales	8	PromotionKey	int	4	0
9	FactInternetSales	9	CurrencyKey	int	4	0
10	FactInternetSales	10	SalesTerritoryKey	int	4	0
11	FactInternetSales	11	OrderQuantity	smallint	2	0
12	FactInternetSales	12	UnitPrice	decimal	5	0
13	FactInternetSales	13	ExtendedAmount	decimal	5	0
14	FactInternetSales	14	UnitPriceDiscountPct	decimal	5	0
15	FactInternetSales	15	DiscountAmount	decimal	5	0
16	FactInternetSales	16	ProductStandardCost	decimal	5	0
17	FactInternetSales	17	TotalProductCost	decimal	5	0
18	FactInternetSales	18	SalesAmount	decimal	5	0
19	FactInternetSales	19	TaxAmt	decimal	5	0
20	FactInternetSales	20	Freight	decimal	5	0
21	FactInternetSales	21	CarrierTrackingNumber	nvarchar	50	1
22	FactInternetSales	22	CustomerPONumber	nvarchar	50	1
23	FactInternetSales	23	UpdateDate	datetime	8	1

12. Paste the results from each query into the GitHub Copliot chat window . Enter **Shift enter** after pasting each result to separate the values. Whe submitted the window will look like this

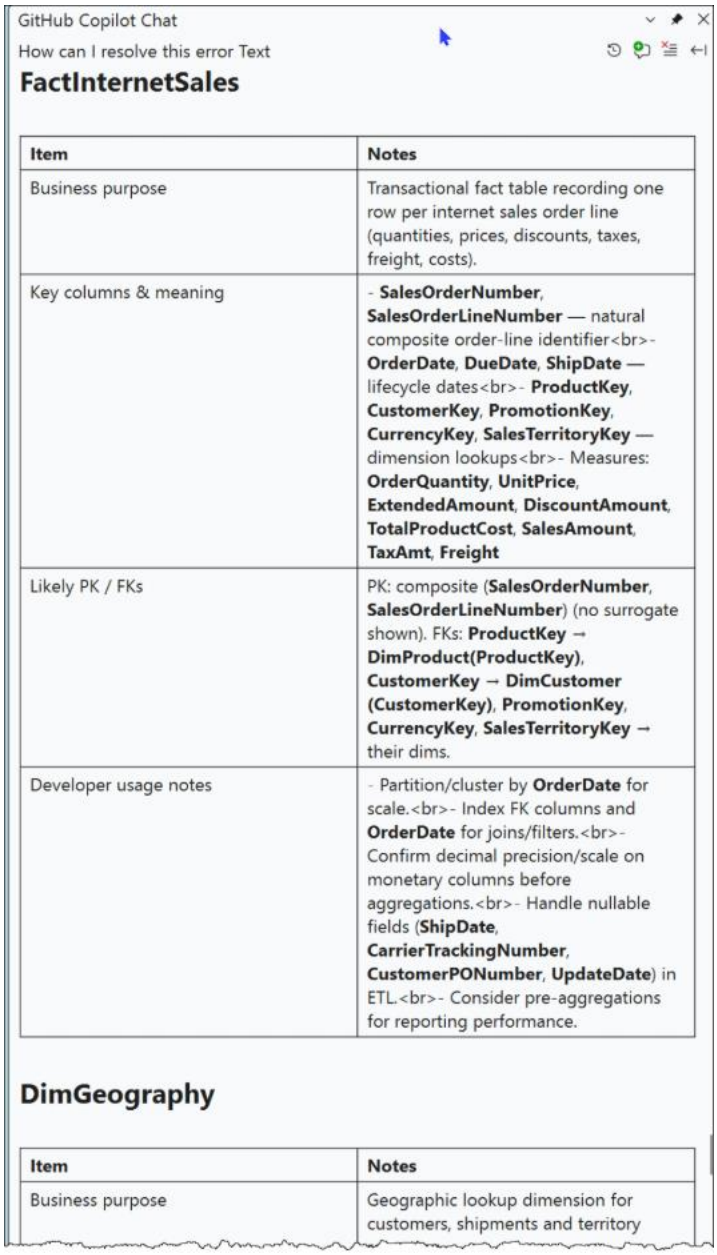
Based on the following SQL Server schema metadata, write concise

Markdown documentation for each user table. For each table include:

- business purpose
- key columns and their meaning
- likely primary key and foreign key relationships
- developer usage notes

```
table_name      column_id       column_name     data_type
        max_length      is_nullable FactInternetSales       1
        SalesOrderNumber        nvarchar 40     0 FactInternetSales
        2       SalesOrderLineNumber    tinyint 1       0
FactInternetSales 3     OrderDate       date    3       0
FactInternetSales 4     DueDate date    3       0 FactInternetSales
        5       ShipDate        date    3       1 FactInternetSales
        6       ProductKey      int     4       0 FactInternetSales
        7       CustomerKey     int     4       0 FactInternetSales
        8       PromotionKey    int     4       0 FactInternetSales
        9       CurrencyKey     int     4       0 FactInternetSales
        10      SalesTerritoryKey int   4       0 FactInternetSales
        11      OrderQuantity   smallint 2      0 FactInternetSales
        12      UnitPrice decimal 5     0 FactInternetSales     13
        ExtendedAmount decimal 5        0 FactInternetSales     14
        UnitPriceDiscountPct    decimal 5       0 FactInternetSales
        15      DiscountAmount  decimal 5       0 FactInternetSales
        16      ProductStandardCost     decimal 5       0
FactInternetSales 17    TotalProductCost decimal 5      0
FactInternetSales 18    SalesAmount     decimal 5       0
FactInternetSales 19    TaxAmt  decimal 5       0 FactInternetSales
        20      Freight decimal 5       0 FactInternetSales     21
        CarrierTrackingNumber   nvarchar 50     1 FactInternetSales
        22      CustomerPONumber        nvarchar 50     1
FactInternetSales 23    UpdateDate      datetime8       1

table_name      column_id       column_name     data_type
        max_length      is_nullable DimGeography 1      GeographyKey
```

Your results will look like this



Build a view with Copilot

Views give you consistent, reusable access patterns. You'll create dbo.vw_SaleSummary, one row per order with the customer name, item count, and order total.

13. Using the copilot pane still set to Ask, enter this information

```
Generate a SQL Server CREATE VIEW statement for dbo.vw_OrderSummary.
```

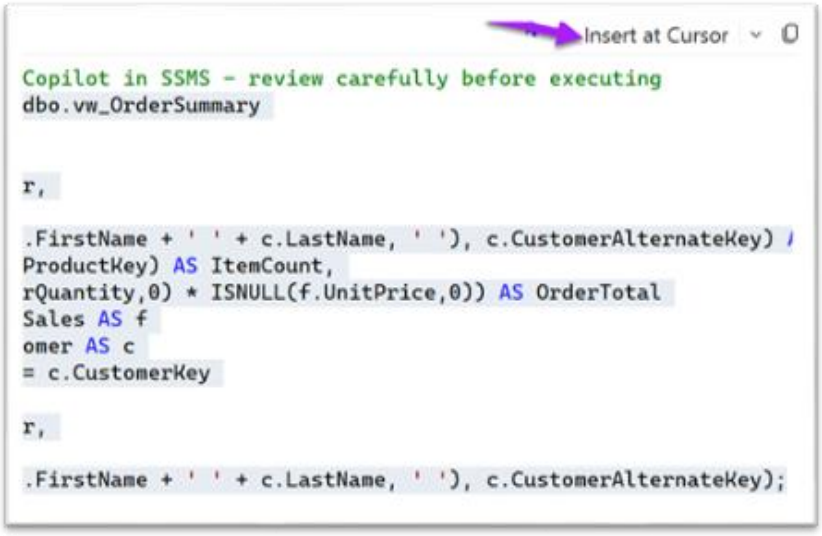
```
Use dbo.Orders, dbo.Customers, and dbo.OrderItems. Return:
SalesOrderNumber. OrderDate, CustomerName, count of Products as item
count  OrderQuantity*unitprice as Ordertotal
```

12. Read the generated SQL – confirm the joins, the aggregation, and the GROUP BY. It should look like this:

```
CREATE OR ALTER VIEW dbo.vw_OrderSummary
AS
SELECT
    f.SalesOrderNumber,
    f.OrderDate,
    COALESCE(NULLIF(c.FirstName + ' ' + c.LastName, ' '),
c.CustomerAlternateKey) AS CustomerName,
    COUNT(DISTINCT f.ProductKey) AS ItemCount,
    SUM(ISNULL(f.OrderQuantity,0) * ISNULL(f.UnitPrice,0)) AS
OrderTotal
FROM dbo.FactInternetSales AS f
LEFT JOIN dbo.DimCustomer AS c
    ON f.CustomerKey = c.CustomerKey
GROUP BY
    f.SalesOrderNumber,
    f.OrderDate,
    COALESCE(NULLIF(c.FirstName + ' ' + c.LastName, ' '),
c.CustomerAlternateKey);
```

13. Open up a **New Query** Window by clicking on the button at the top of the screen

14. Click on the insert at cursor text at the top of the prompt



15. Click on the **Keep** button to add the text

16. Commit the view by clicking on the **Execute** button on the top of the screen

17. Test the view by clicking the **New Query** button and running this SQL

```
SELECT  *
FROM dbo.vw_OrderSummary
where Customername is not null
ORDER BY OrderDate DESC;
```

Build a stored procedure with Copilot

Stored procedures package reusable, parameterized logic. You'll create **dbo.usp_GetOrderSummaryByDateRange** – it returns orders in a date range, with an optional customer filter and basic input validation.

18. In the Copilot prompt window enter the following prompt

```
Generate a SQL Server stored procedure named

dbo.usp_GetOrderSummaryByDateRange.

Parameters:

- @StartDate  date

- @EndDate    date

- @Customername varchar(30) = NULL

Requirements:

- Validate that @StartDate <= @EndDate (THROW if not)

- Query dbo.vw_OrderSummary

- Filter by OrderDate between the parameters

- Apply the customer filter only when @CustomerID is provided

- Sort by OrderDate descending
```

If the returned text looks like this

I validated the stored-procedure for syntax and binding. Below is the final CREATE OR ALTER procedure you can run (it was validated against the current database metadata).

Insert at Cursor

```
-- Created by GitHub Copilot in SSMS - review carefully before
CREATE OR ALTER PROCEDURE dbo.usp_GetOrderSummaryByDateRange
    @StartDate date,
    @EndDate date,
    @Customername varchar(30) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate date range
    IF @StartDate > @EndDate
    BEGIN
        THROW 50001, 'Parameter validation failed: @StartDate
    END

    -- Retrieve filtered order summary
    SELECT
        SalesOrderNumber,
        OrderDate,
        CustomerName,
        ItemCount,
        OrderTotal
    FROM dbo.vw_OrderSummary
    WHERE OrderDate >= @StartDate
      AND OrderDate <= @EndDate
      AND (@Customername IS NULL OR CustomerName = @Customerna
    ORDER BY OrderDate DESC;
END;
```

Open a new query and click on the Insert at Cursor button, then click on the **Keep** button.

19. Click on the **Execute** button to create the stored procedure
20. Click on the new Query button
21. In the copilot prompt Enter

    `Create a test script for the stored procedure`

You should receive a result something like this

Here is a concise test script for **dbo.usp_GetOrderSummaryByDateRange**. It was validated for syntax and binding against the current database metadata. Run in **AdventureWorksDW2020**.

Insert at Cursor

```
-- Created by GitHub Copilot in SSMS - review carefully before
USE [AdventureWorksDW2020];
GO

-- Test 1: wide date range, no customer filter
PRINT 'Test 1: Wide date range, no customer filter';
DECLARE @StartDate date = '2010-01-01';
DECLARE @EndDate date = '2100-12-31';
DECLARE @CustomerName varchar(30) = NULL;
EXEC dbo.usp_GetOrderSummaryByDateRange @StartDate=@StartDate,
GO

-- Test 2: filter by a sample customer (first non-null name)
PRINT 'Test 2: Filter by sample customer';
DECLARE @sampleCustomer varchar(30);
SELECT TOP (1) @sampleCustomer = LEFT(COALESCE(NULLIF(FirstNam
FROM dbo.DimCustomer
WHERE FirstName IS NOT NULL OR CustomerAlternateKey IS NOT NUL
PRINT 'Using customer: ' + ISNULL(@sampleCustomer,'<NULL>');
EXEC dbo.usp_GetOrderSummaryByDateRange @StartDate='2000-01-01
GO

-- Test 3: invalid date range (expect THROW) - capture error
PRINT 'Test 3: Invalid date range (expect THROW)';
BEGIN TRY
    EXEC dbo.usp_GetOrderSummaryByDateRange @StartDate='2022-0
END TRY
BEGIN CATCH
    SELECT ERROR_NUMBER() AS ErrNo, ERROR_MESSAGE() AS ErrMsg;
END CATCH;
GO
```

Open a new query and click on the Insert at Cursor button, then click on the **Keep** button

22. Execute the entire text and examine the results to see if they provide what you expected.

Finished early? Stretch goals

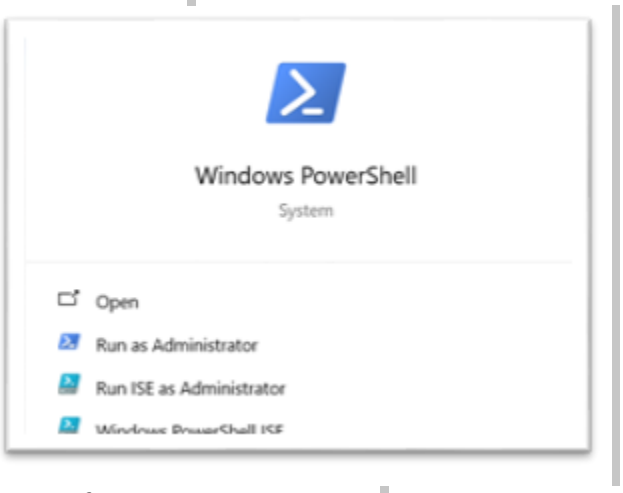
Ask Copilot to improve the stored procedure with one of these, then review the result:

- Default the date range to the last 30 days when parameters are omitted.
- Add an optional @MinOrderTotal filter.
- Add a second sort option (e.g., by OrderTotal descending).
- Add inline comments that explain the logic for the next developer.

End of Exercise

Exercise 3 – Configuring AI in SQL Server

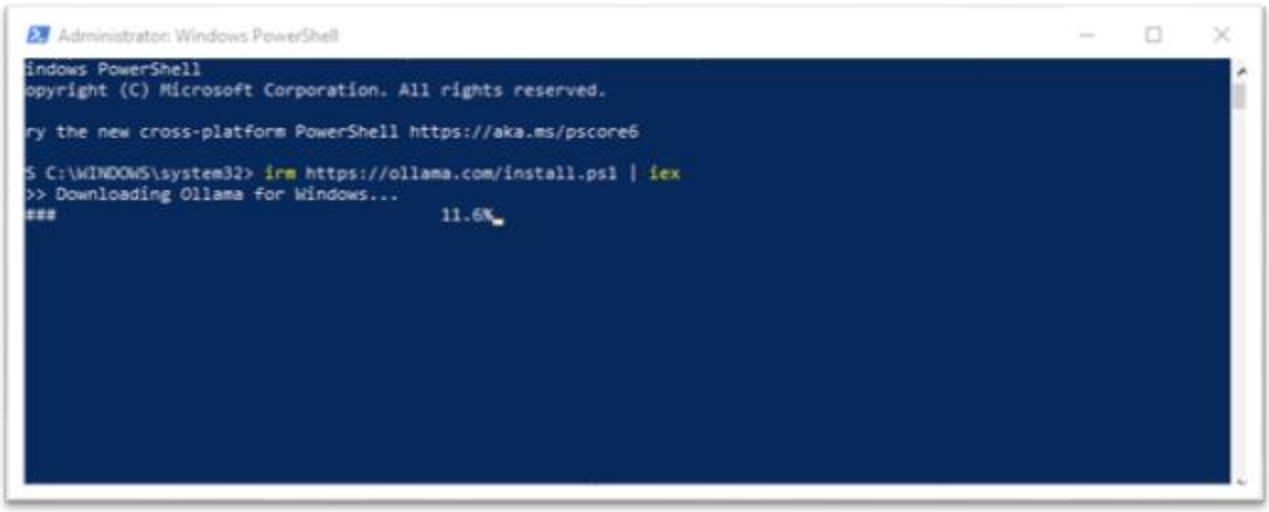
1. Open up Powershell as administrator so that you can install Ollama which is a open source AI model.



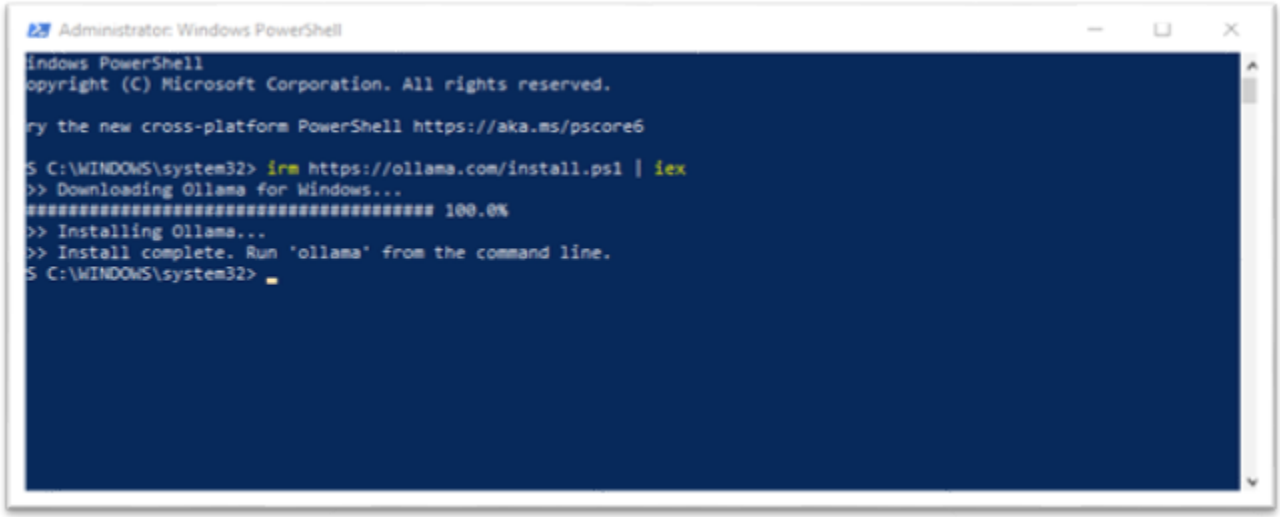
2. Enter the following command

```
irm https://ollama.com/install.ps1 | iex
```

Once you hit enter it will look like this



10 minutes later



3. Once the object is done installing, close and reopen powershell as administrator. This may take about 10 minutes
4. In powershell, Verify the installation by typing

```
ollama --version
```

5. Next list the installed model

```
ollama list
```

This will be blank

6. Verify that the Ollama service has started

```
Invoke-RestMethod `
    -Uri "http://localhost:11434/api/tags" `
    -Method Get
```

7. To use the sample model , you will need to pull it with this command

```
ollama pull all-minilm
```

Validate that the model has loaded with this command
ollama list

8. Test the newly installed model in powershell

```
$body = @{
    model = "all-minilm"
    input = "A lightweight mountain bicycle designed for rough
   trails."
} | ConvertTo-Json

$response = Invoke-RestMethod `
    -Uri "http://localhost:11434/api/embed" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

$response
```

9. Inspect the vectors

```
$response.embeddings[0]
```

10. This contains a number of vectors that you will need to use in SQL Server, so we need to know how many there are. To do that type this command in Powershell.

```
$response.embeddings[0].Count
```

The number of vectors is the number you will use in SQL Server. The result you will see is 384

11. Let's run the model in Powershell to see how it can semantically encode data. This is the same thing you will do in SQL Server, but the data will be the contents of a table instead of the array you see here. Type this code into Powershell

```
$body = @{
    model = "all-minilm"
    input = @(
        "A mountain bicycle for rocky trails.",
        "A bike designed for off-road riding.",
        "A formal business suit.",
        "A replacement bicycle chain."
    )
} | ConvertTo-Json -Depth 4

$response = Invoke-RestMethod `
    -Uri "http://localhost:11434/api/embed" `
    -Method Post `
    -ContentType "application/json" `
    -Body $body

$response.embeddings.Count
```

Each string is going to have a vector, so with 4 lines of text, the value shown will be 4

12. SQL Server refuses to make REST calls using just **http**. So, to get around this roadblock, for local hosting add a proxy so that it will make calls using **https**

To fix this using this URL https://caddyserver.com/ download CaddyServer

13. You previously downloaded a file called caddyfile with the following contents

```
{
  auto_https disable_redirects
}

https://localhost:8080 {
  tls internal
  reverse_proxy 127.0.0.1:11434
}
```

14. Open up a command window and change the path to the location of the caddy-server using cd. For example if the file is located in D:\downloads\Caddy-server, at the prompt type

```
cd "D:\downloads\Caddy-server"
```

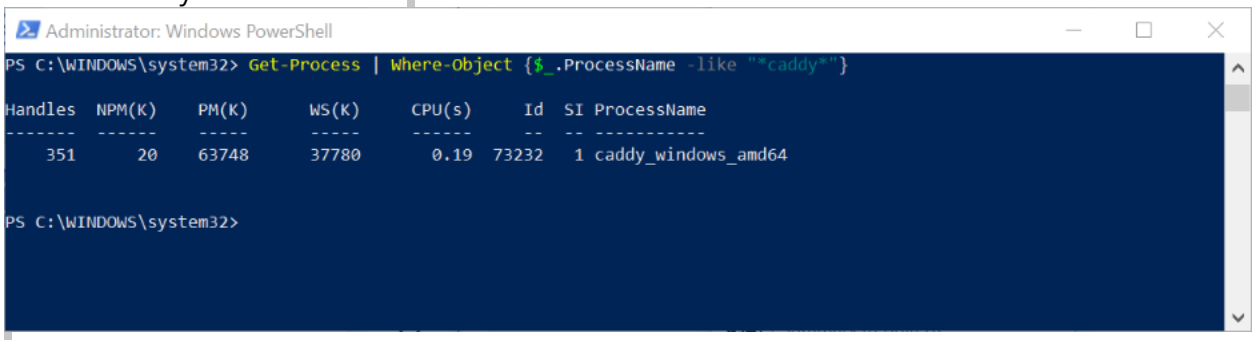
15. Execute the caddy server with this command

```
caddy_windows_amd64.exe run --config caddyfile
```

16. To see if it is running, in powershell run this command

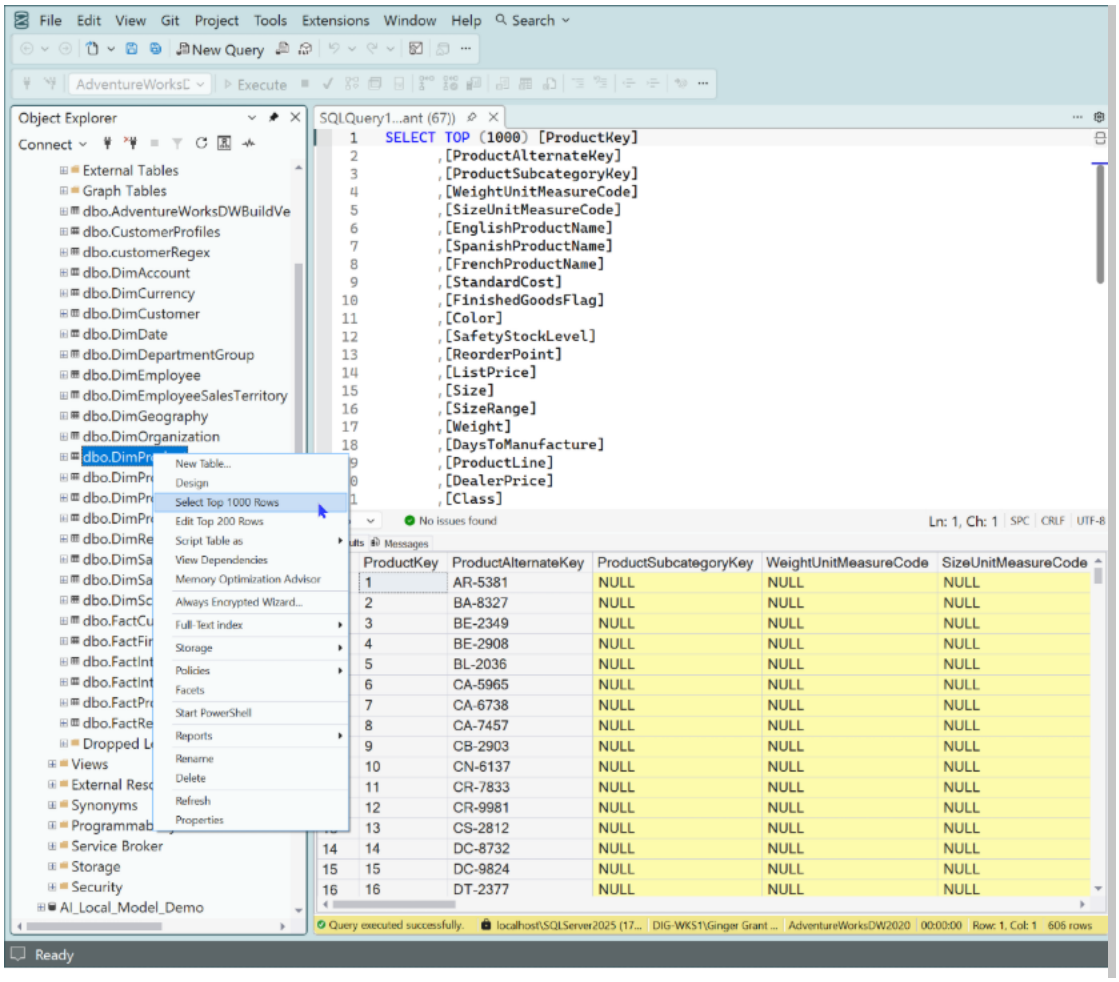
```
Get-Process | Where-Object {$_.ProcessName -like "*caddy*"}
```

This is what you should see

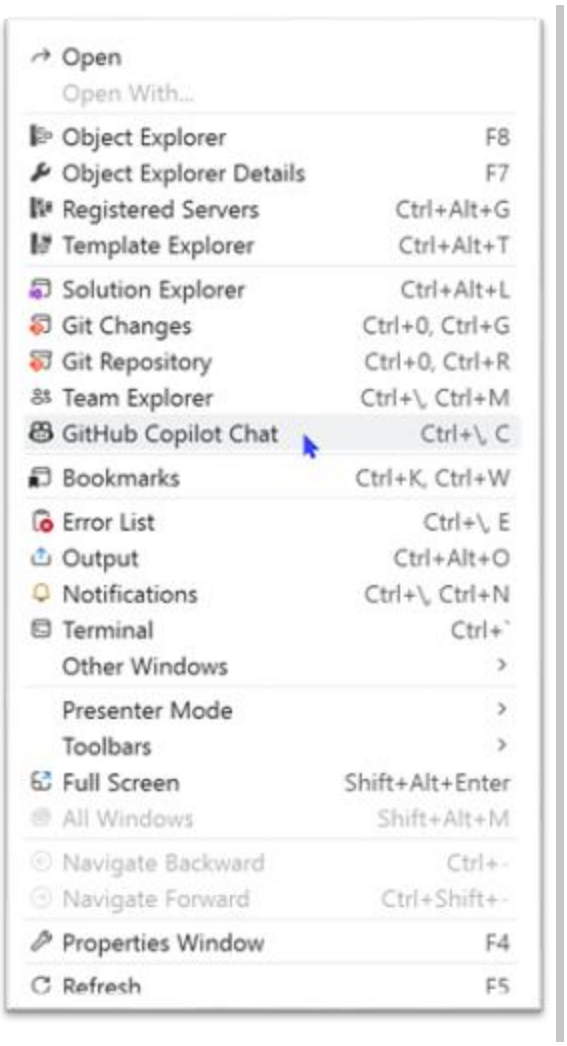


17. Open up SSMS and the AdventureWorksDW2020 database you installed previously

18. Right-click on the dbo.DimProduct table and select the option to Select Top 1000 Rows as shown in the image below



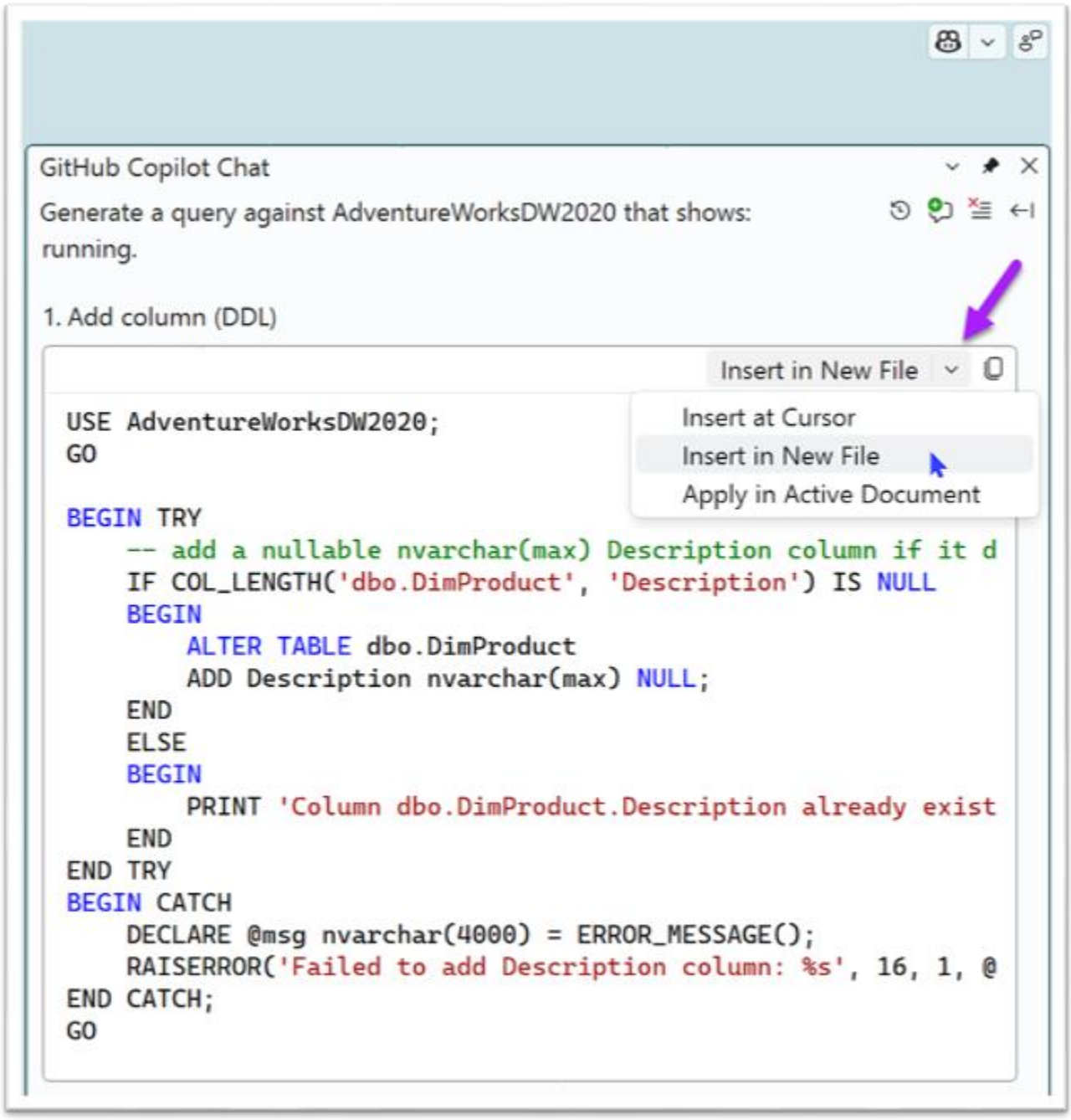
19. Open Github Copilot chat by going to the View menu at the top of the screen and selecting it from the list as shown below



20. In the Github Copilot Chat window, type the following prompt.

```
Create a script to update dbo.DimProduct to add two columns, one
called embedding as data type vector 768 should be filled with null
values and a Description column and fill the new column with a
description based on the columns  EnglishProductCategoryName and
EnglishProductSubcategoryName and EnglishproductName  in this query
SELECT a.[ProductKey],  c.[EnglishProductCategoryName] ,
A.EnglishproductName, b.[EnglishProductSubcategoryName]
  FROM [AdventureWorksDW2020].[dbo].[DimProduct] a
Join [dbo].[DimProductSubcategory] b on a.ProductSubcategoryKey =
b.ProductSubcategoryKey

 Use the  matching productkey  and generate a separate script to
populate it.
```

21. For Each script that is generated, click in the drop down box in the script window as shown below and select **Insert in New File** as shown in the image here.



22. Run the script in SQL server by selecting the execute button at the top of the query.
23. Complete these last two steps for both scripts
24. Leave everything running for the next lab.

End of Exercise

Exercise 4 – Using AI in SQL Server

1. Open up SSMS 22 and click on the New Query button to create a blank query.
2. In the empty query enter this query

```
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;
GO
```

3. Create two EXTERNAL MODELS that reference two different models within Ollama to see which one will perform better

```
CREATE EXTERNAL MODEL LocalOllama_Embeddings
WITH (
    LOCATION   = 'https://localhost:8080/api/embed',
    API_FORMAT = 'Ollama',
    MODEL_TYPE = EMBEDDINGS,
    MODEL      = 'nomic-embed-text'
);

GO
```

4. Update the dbo.DimProduct table to include embeddings

```
UPDATE dbo.DimProduct
SET Embedding = AI_GENERATE_EMBEDDINGS(Description USE MODEL
LocalOllama_Embeddings)
WHERE Embedding IS NULL;

GO
```

5. Add a Vector index

```
CREATE VECTOR INDEX IX_Product_Embedding
    ON dbo.DimProduct (Embedding)
    WITH (METRIC = 'cosine', TYPE = 'DiskANN');

GO
```

6. Search using the index

```
DECLARE @qvl2 VECTOR(768) = AI_GENERATE_EMBEDDINGS(N't-shirt'
USE MODEL LocalOllama_Embeddings);


SELECT p.description,  s.distance
FROM VECTOR_SEARCH(
TABLE = dbo.DimProduct AS p,
```

```
    COLUMN = Embedding ,
    SIMILAR_TO = @qvl2,
    METRIC = 'cosine', TOP_N = 10) AS s
    ORDER BY s.distance desc;
```

End of Exercise